CSC209: Final Project Report

Sylvia Wong, Raiyan Tanvir Ta-seen, Yifan He

Project Overview

Our team created an image processing application, where image filters are applied in sequence to a bitmap image to produce a processed image. The filters in our application include greyscale, vignette, emboss, sharpen, glitch effect, gaussian blur, pixelate, edge detection using Laplacian operator, invert, and black and white. Out of these filters, filters implemented with a kernel include sharpen, emboss, gaussian blur, and edge detection. Filters that are implemented by performing a calculation on a row of pixels include grayscale, glitch effect, pixelate, invert, vignette, and black and white. When running the program, the user inputs the name of the bitmap image file, the number of filters the user would like to apply, and the specific filters the user wants to apply in the order in which the user wants to apply them. If the filters require a strength (i.e. vignette, glitch effect, gaussian blur, pixelate, black and white), the user enters the strength along with the filter name (e.g. glitch_effect75). Then, the program will apply the filters to the bitmap image and produce a bitmap file whose name indicates the filters applied. For example, if the user wants to process an image called test.bmp with the filters vignette with a strength of 50, sharpen, and glitch effect with a strength of 25 in this order, the user's input would be the following: ./main test.bmp 3 vignette50 sharpen glitch_effect25. The program will produce a bitmap image called test_vignette50_sharpen_glitch_effect25.bmp containing the processed image.

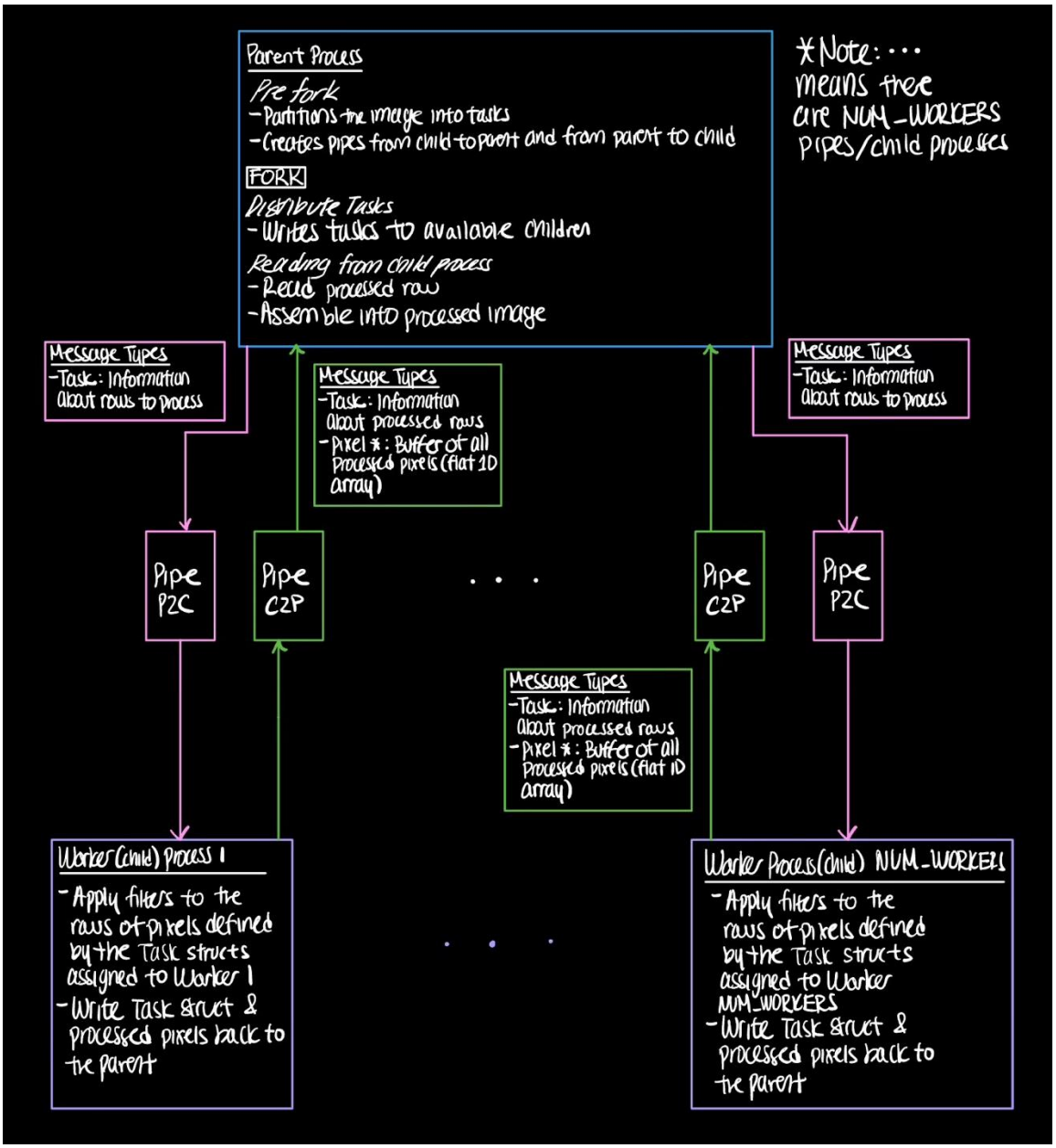
Our application is part of category one as the program uses pipes to communicate subsections of the image between the parent to multiple child processes. In our project, we use an architecture like the worker pool architecture. Specifically, the parent divides the bitmap image into subsections, known as "tasks" in this report, defined by the row in the image that the subsection starts at and the row that it ends at. It then sends these tasks to the child processes that are not currently processing another task. The child processes who receive a task then apply filters to their assigned subsection and write the processed data back to the parent for the parent to reassemble into a processed image (see section Concurrency Model for more details). This design achieves concurrency between the worker processes and between the children and the parent which speeds up the image processing as different sections of the image are being processed at the same time. Ultimately, it makes image processing more efficient and effective.

Build Instructions

- To compile the program:
    - Command: make
- Input Files:
    - Files required: a (24-bit or 32-bit) .bmp (bitmap) image file in the same directory as the program (Must have more rows than NUM_TASKS, which is currently 48.)

    - Example: test.bmp, dog.bmp, etc.
- To run the program (user input):
    - Command line arguments: bitmap image to be processed, how many filters to apply, the filters in the order to be applied with filters requiring strength formatted as filterX (where the strength, X, is a number from 1-100)
    - Command (general): ./main <input image> <number of filters> <filter 1> ... <filter n>
    - Example: ./main test.bmp 3 grayscale emboss glitch_effect50

Architecture Diagram



Communication Protocol

Field	Description
Direction	Parent → Worker

Encoding	A binary struct Task is written to each worker process by a write_all call, a helper function that prevents partial reads by calling write until all bytes are written.
	
	The write_all helper function is defined in lines 37-46 in process.c
	
	The Task struct is defined as follows:
	typedef struct {
	int task_id; // track which block we are on
	int start_row; // where to start the block
	int end_row; // where to end the block (exclusive)
	} Task;

Semantics	Instructs the worker to process a new task. The worker knows how many bytes to read because the parent always sends a fixed-size Task struct, so the worker calls read_all with sizeof(Task) to ensure the entire message is received. The struct is stored in a local Task variable. The message contains information about which row the worker process should start operating on and which row it should stop at. The worker process reads the struct, applies the filters requested by the user to all the rows from start_row to end_row (excluding end_row) and writes the processed rows and task information back to the parent. To ensure correctness when applying multiple filters, the worker also loads and processes additional "halo rows" around the assigned task. However, only the rows from *start_row* to end_row are returned to the parent.
Error Handling	If write_all returns -1 or 0, an error message is displayed using perror and the program is terminated using exit(1). The program is terminated because a processed image cannot be generated if a write call of processed pixels fails. write_all returning 0 would indicate that it failed as we expect to write sizeof(Task) bytes. If read_all, defined similarly to write_all in lines 26-35, returns a number of bytes different from the number of bytes written to the pipe, an error message is displayed using perror and the program is terminated using exit(1). The program is terminated because the worker doesn't know which rows to process if a Task is not fully read from the pipe. Note that write_all & write and read_all & read have the same return values (i.e. -1 for failure, 0 for no bytes written, n for n bytes written)

Field	Description
Direction	Worker → Parent

Encoding	A binary struct Task is written to the parent process by a write_all call and a buffer of all the processed pixels required by Task is written to the parent process by a write_all call.
	
	The write_all helper function is defined in lines 37-46 in process.c
	
	The Task struct is defined as follows:
	typedef struct {
	  int task_id; // track which block we are on
	  int start_row; // where to start the block
	  int end_row; // where to end the block (exclusive)
	} Task;
	
	The pixel struct is defined as follows:
	typedef struct pixel {
	  unsigned char blue;
	  unsigned char green;
	  unsigned char red;
	  unsigned char alpha;
	} pixel;
	
	The number of bytes written in a write_all call for the buffer of processed pixels is (task.end_row – task.start_row) * width * sizeof(pixel).

Semantics	Task: Instructs the parent that the child is completed. The Task struct contains information about the rows that the worker processed. The parent reads the Task struct then reads the buffer (see below). Buffer: Provides the parent with processed pixels. The buffer contains processed pixels from the worker's assigned Task. The parent reads the buffer of processed pixels and copies the pixels into *processed_pixels* using the Task struct to determine where to copy it (lines 390-415).
Error Handling	If write_all returns -1 or 0, an error message is displayed using perror and the program is terminated using exit(1). The program is terminated because a processed image cannot be generated if a write call of processed pixels fails. Write_all returning 0 would indicate that it failed as we expect to write sizeof(Task) bytes in the first call and width (task.end_row – task.start_row) * sizeof(pixel) bytes in the second call. If read_all returns a number of bytes different from the number of bytes written to the pipe, an error message is displayed using perror and the program is terminated using exit(1). The program is terminated because the

	parent cannot assemble a processed image without reading all the data from the pipes. Note that write_all and write have the same return values (i.e. -1 for failure, 0 for no bytes written, n for n bytes written)

Concurrency Model

When the parent forks

In our program, the function process_filters, which handles pipe communication, is called after the bitmap image is read (see bitmap.c). The parent then partitions the image into *NUM_TASKS* tasks (see lines 90-109 in process.c) and creates pipes (see lines 116-126 in process.c) for each worker process using separate 2D file descriptor arrays: one from the parent to the child and one from the child to the parent. After creating all the pipes, the parent forks *NUM_WORKERS* workers in a loop to process the tasks concurrently (see line 128-130 in process.c). Once the parent forks the workers, it initially distributes one task to each worker for the workers to process concurrently and continues to distribute tasks to workers that are available, meaning they have completed their previously assigned task. This is achieved using the select system call, which allows the parent to monitor all worker pipes simultaneously and detect which worker has completed its task. (see lines 305-364 and 417-432 in process.c).

Worker processing

When a worker is processing its assigned task, it first loads additional "halo rows" around the assigned block to ensure correctness when applying multiple filters sequentially. The halo size is equal to the number of filters, allowing each stage of filtering to access the required neighboring rows without being affected by previously modified data. The worker then applies the filters row by row using a sliding window of three rows at a time (see lines 158–265 in process.c), since some filter algorithms use a 3x3 kernel to determine a pixel's RGB based on its eight neighboring pixels. For example, gaussian blur uses a 3x3 matrix to determine the RGB of the pixel in the middle of the kernel and it requires the neighboring pixels to do so. All worker processes process their tasks concurrently which makes the image processing more efficient compared to processing the whole image at once.

How parent knows when the workers are ready

While the workers are processing their assigned tasks, the parent uses the select system call to monitor active workers for incoming data. Rather than blocking on a single worker using read_all, select tells the parent which child processes are ready with processed data. When a worker signals completion, the parent can call read_all immediately to collect the processed data and assign a new task to the worker. This helps ensure that the worker processes remain active to process more tasks and prevents the parent from blocking unnecessarily. After collecting processed pixels from the completed worker processes, the parent begins reassembling the

processed pixels into an array, and after all the workers are finished processing all the tasks, the parent writes the final output to the output file (see lines 452-455 in process.c). The logic with select, read, and reassemble is seen in lines 305-435 in process.c.

How parent collects child processes

The parent collects child processes by calling waitpid on each worker process to reap the worker processes after they have exited (see lines 442-450 in process.c). Waitpid is called on each id in the ids array, and nothing is done with the exit value of each worker process. This ensures that all child processes are properly reaped and prevents zombie processes from accumulating.

Error Handling and Robustness

Worker or Parent writing to a closed pipe

In our program, it could be possible that a process could attempt to write to a closed pipe if pipes were not properly closed in the code. When a process attempts to do this, write_all will return -1. So, the program contains an if statement that checks for this case, uses perror to print an error message to the terminal explaining why the write call failed, and immediately terminates with an error code of 1 (see lines 248-255 in process.c). The program terminates because a processed image cannot be assembled if the processed rows of the image are not properly written to the parent.

User inputs wrong command line arguments

In our program, we require the user to input the name of the bitmap image file, the number of filters the user wants to apply, and the filters in the order to be applied with strength indicated if applicable (see build instructions). If a user forgets one of these arguments or enters the wrong argument, the program handles this by printing a message to standard error saying: “usage: main <input image> <number of filters> <filter 1> ... <filter n>”. If the user enters a filter that does not exist, the program warns the user with the following message: “Filter X is not yet implemented”. If the user enters an invalid filter number, the program warns the user by printing the following message to standard error: "Error: number of filters to apply must be a positive integer and less than 100 inclusive". Finally, if the user forgets to indicate strength for filters that require a strength factor, the program prints the following message to standard error: “filter_x requires strength from 1 to 100, for example filter_x50”. These error checks help keep the program robust and remind the user of the correct format for the command line arguments (see lines 64-97 and 112-124 in main.c).

Parent fails to read full buffer from a worker

In our program, it could be possible that the parent to fails to read the full buffer from a worker process either because it attempts to read from a closed pipe or because it performs a partial read. Our program handles these cases by checking whether the return value of the read_all

call is not equal to the number of bytes written to the pipe in the worker process. If they are not equal, an error message is printed using perror and the program is immediately terminated with an exit code of 1 (see lines 397-406 in process.c). Furthermore, our program prevents the partial reads from occurring because we implemented a function called read_all (see lines 26-35 in process.c). This function calls read repeatedly until all the bytes are read from the worker class to prevent partial reads.

As an additional note, we created a helper function called safe_close (see lines 48–54 in process.c) to safely close file descriptors while ignoring errors caused by attempting to close an already closed descriptor (e.g., EBADF).

Team Contributions

Sylvia

In this project, I programmed the filters grayscale, emboss, sharpen, and vignette. The emboss and sharpen filters require the sliding window technique as they depend on neighboring pixels to calculate the RGB. For the pipeline and concurrency, I had experimented with a pipe design where a child was created for each filter in sequence and information was piped through these filters. Although this design was infeasible, the design phase was crucial to understand how the workers and parent should interact. I also modified Raiyan's initial pipe design to implement a more complex architecture. In this design, I use a worker pool and distribute subsections of the image to the workers in a round robin fashion. This requires the parent to communicate the tasks to the children and the child to communicate the processed data back to the parent for the parent to reassemble into a final image. Outside of the program, I attended office hours on behalf of the team to get advice on our project, and I wrote this report describing our application.

Raiyan

I programmed the filters glitch effect, gaussian blur, edge detection using Laplacian operator, and pixelate. All these filters depend on kernel operations, which heavily utilize our "sliding window" technique in order to pass in the required neighbouring pixels. I also programmed the parsing of the command line argument as well as the code to load in the image including the pixel structs which is similar to the one used in Lab 6. For the main concurrency part, I designed one of the initial model to which Sylvia added on the idea of using a "worker pool" in order to introduce more complex intercommunication between the parent and child instead of the parent just wating for a child to finish their row block. As well, I came up with the row block idea since I was initially having some issues with opening too many pipes, or forking too many child processes at first.

Yifan

In this project, I programmed the filters "invert" and "black and white". I originally implemented 8 filters, but they used the same communication strategy and did not contribute

much to the project's focus on pipes, so I kept only two. I also added alpha preservation code to all filters so that 32-bit BMPs do not lose their alpha channel during processing.

For the pipeline setup, I simplified and fixed issues in the original design by introducing the helper function "safe_close" to safely close file descriptors while intentionally ignoring "repeated close errors" (e.g. EBADF). This helped make the pipeline setup easier to understand.

For concurrency, I implemented the current "parent waits for a child to finish before assigning the next task" approach using "select()", which improves performance compared to the previous round robin design where the parent assigned all tasks at once at the beginning. This allows better handling when some workers are much slower than others. I also improved the row-splitting logic so that remainder rows are distributed evenly across tasks instead of being assigned entirely to the last.

Additionally, I implemented the current "halo" approach to maintain correctness under concurrency. There was a major issue in the original design where neighboring rows would be overwritten after one filter, making them unusable for subsequent filters.

Finally, I added error checking for system calls and OS-related library calls, as well as checks for invalid or empty parameters. I also implemented checks for input files, ensuring they are 24-bit or 32-bit BMPs, and added a check to ensure the image height is sufficient for processing.

Outside the program, I also went to office hours constantly to discuss pipeline and concurrency designs with the professors.